

FlowMLLab: An open-source framework for reproducible computational-fluid-dynamics and scientific-machine-learning experiments

Ehsan Roohi^{a,*}

^a*Department of Mechanical and Industrial Engineering, University of Massachusetts Amherst, 160 Governors Drive, Amherst, MA 01003, USA*

Abstract

FlowMLLab is open-source educational and research software for a first rigorous encounter with artificial intelligence (AI) in thermal-fluid engineering. It was used in the Summer 2026 graduate course *AI in Fluid Mechanics* at the University of Massachusetts Amherst and is also intended as an onboarding path for engineers entering industry. The software emphasizes validated numerical data, scaling, case-wise splitting, transparent baselines, multilayer perceptrons, proper orthogonal decomposition (POD), and restricted neural operators before introducing research-level kinetic closures in the final notebooks. One lid-driven cavity connects computational fluid dynamics (CFD), direct simulation Monte Carlo (DSMC), and reduced-order modeling. On three blind Reynolds-number cases, the maximum velocity error is 0.4941% for POD–Galerkin and 0.6338% for POD–DEIM (Discrete Empirical Interpolation Method); the latter provides a measured $9.24\times$ online speedup and a 7.84-query break-even count. Existing gates retain blind POD–DeepONet errors below 0.5% and validate DSMC wall pressure against published data to 0.772%. Seventeen notebooks, five lecture PDFs, versioned evidence, command-line tools, and Python 3.10–3.12 tests connect every scientific claim to executable software.

Keywords: open-source software, scientific machine learning, thermal-fluid education, POD–DEIM, direct simulation Monte Carlo, reproducibility

*Corresponding author

Email address: `roohie@umass.edu` (Ehsan Roohi)

Required Metadata

Current code version

Nr.	Code metadata description	Current software record
C1	Current code version	FlowMLLab 1.1.0 (Git commit 04b6c036fcc2 ; version DOI 10.5281/zenodo.22126785)
C2	Permanent GitHub link to code/repository used for this code version	tagged source and release record
C3	Legal Code License	Massachusetts Institute of Technology (MIT) License
C4	Code versioning system used	Git
C5	Software code languages, tools, and services used	Python; Jupyter Notebook; NumPy; SciPy; pandas; Matplotlib; scikit-learn; TensorFlow/Keras; optional Compute Unified Device Architecture (CUDA) and CuPy
C6	Compilation requirements, operating environments & dependencies	Python 3.10–3.12; Linux, macOS, Windows or Google Colab; installation through <code>pyproject.toml</code> ; optional TensorFlow and CUDA for neural or Fokker–Planck applications
C7	Developer documentation/manual	START_HERE guide; course map ; figure map
C8	Support email for questions	roohie@umass.edu

Table 1: Code metadata (mandatory).

1. Motivation and significance

Machine learning supports reduced-order modeling, closure construction, flow reconstruction and solver acceleration [1, 2, 3]. Many demonstrations nevertheless begin with prepared arrays and evaluate random points from the same fields, obscuring numerical provenance and permitting leakage between physical cases. A neural model may then appear useful even when interpolation is more accurate. FlowMLLab instead makes simulation and data qualification explicit software stages; selection precedes blind testing, and physical validation closes the workflow.

The primary goal is education for graduate engineering students and practicing engineers who know fluid mechanics but may have little prior Python or machine-learning experience. FlowMLLab was used to teach MIE 690A, *AI in Fluid Mechanics*, at the University of Massachusetts Amherst in Summer 2026. It deliberately teaches essential ideas through bounded models rather than presenting an encyclopedic collection of advanced architectures. Numerical provenance, nondimensionalization, scaling, losses and baselines come first; operator learning and learned kinetic closures are reserved for the later weeks after users can audit the physical data and explain a blind test. This sequence also makes the software a practical first AI experience for an engineer moving into industry.

The continuum benchmark is a two-dimensional lid-driven cavity with side length L and lid speed U_{lid} . Dimensional position and time are written as (x^*, y^*) and t^* , with nondimensional counterparts $x = x^*/L$, $y = y^*/L$ and $t = t^*U_{lid}/L$. Velocity is scaled as $u = u^*/U_{lid}$ and $v = v^*/U_{lid}$, while streamfunction ψ and vorticity ω use the scales $U_{lid}L$ and U_{lid}/L , respectively. For density ρ and dynamic viscosity μ , the Reynolds number is $\text{Re} = \rho U_{lid}L/\mu$, and the streamfunction–vorticity solver advances

$$\nabla^2 \psi = -\omega, \quad (1)$$

$$\frac{\partial \omega}{\partial t} + u \frac{\partial \omega}{\partial x} + v \frac{\partial \omega}{\partial y} = \frac{1}{\text{Re}} \nabla^2 \omega, \quad (2)$$

where $u = \partial \psi / \partial y$ and $v = -\partial \psi / \partial x$, and the Laplacian ∇^2 acts on the nondimensional coordinates. Because the velocity definition enforces incompressibility identically, the finite-difference solution can be audited through its residuals and compared directly with accepted velocity benchmarks [4].

Pressure is recovered after the velocity solution. Defining $p = (p^* - p_{ref}^*)/(\rho U_{lid}^2)$ relative to the dimensional reference pressure p_{ref}^* , the momentum equations provide

$$p_x = -(uu_x + vu_y) + \text{Re}^{-1} \nabla^2 u, \quad (3)$$

$$p_y = -(uv_x + vv_y) + \text{Re}^{-1} \nabla^2 v, \quad (4)$$

where subscripts denote partial differentiation, so that $u_x = \partial u / \partial x$ and $p_y = \partial p / \partial y$. A sparse least-squares system fits the edge gradients, and the gauge $p(1/2, 1/2) = 0$ removes the arbitrary pressure constant. Pressure becomes an admissible learning target only after this reconstruction has been compared with the Botella–Peyret benchmark [5].

The second pathway introduces Maxwellian sampling and a compact DSMC cavity solver based on the hard-sphere no-time-counter (HS–NTC) method.

Particle moments expose rarefaction and sampling noise, while time averaging distinguishes stochastic variation from a persistent signal. Published wall-pressure data validate the implementation [6, 7], after which a neural collision-operator study supplies the research-scale comparison [8]. Thus, one evidence hierarchy covers deterministic continuum fields and stochastic kinetic estimates.

FlowMLLab serves researchers and instructors who need an inspectable reference implementation rather than disconnected demonstrations. Users may run the complete suite or replace one solver, while a new learner is evaluated against fixed evidence under the same protocol. Notebooks document the application programming interface (API) and provide examples, but the tested Python package remains the product. FlowMLLab thereby complements notebook-centered narratives [9, 10] while implementing reproducible-computing recommendations [11, 12] through shared modules for physical splitting, baseline assessment and failure diagnosis.

2. Software description

2.1. Software architecture

The repository has five interacting layers. The installable `flowmlab/` package handles discovery, asset checks, figure generation, cavity ROMs and quality assurance (QA), whereas `common/` contains the CFD solver, pressure recovery, physical partitions and baselines. The `data/` and `results/` directories separate reference evidence from predictions, while `advanced/` isolates Fokker–Planck calculations from the core. POD–DeepONet, POD–Galerkin, POD–DEIM and DSMC share metric definitions, so continuum and kinetic comparisons follow the same conventions. Seventeen notebooks expose these components as tutorials rather than independent implementations.

GitHub Actions installs the package on Python 3.10–3.12 and tests numerical utilities, file contracts, hashes and boundary conditions. Notebook execution and Portable Document Format (PDF) inspections complete the release process, covering both machine-readable outputs and user-facing material.

Fields are qualified before learning, and complete physical cases enter development, validation or blind subsets without splitting their grid points. Blind cases remain closed during model choice, while interpolation precedes each neural model. Aggregate errors accompany local diagnostics: continuum tests inspect centerlines and topology, whereas particle tests also require replicate statistics and higher moments. These rules allow new models to use identical evidence.

2.2. Data contract and reproducible execution

The cavity archive stores u , v , p , ψ and ω on a 65×65 grid for Reynolds numbers $\{100, 150, 175, 200, 225, 250, 275, 300, 350, 375, 400\}$. A quality table records solver acceptance, and a Secure Hash Algorithm 256-bit (SHA-256) digest detects substitution. Coordinates are scaled by L , velocity by U_{lid} and pressure by ρU_{lid}^2 .

The splitting unit is a complete Reynolds-number field, so nodes from one case never cross subset boundaries. Blind cases cannot influence scaling, architecture selection, POD rank or early stopping, and loss weights are fixed before blind evaluation. Shared utilities enforce these restrictions.

Comma-separated-values (CSV) files preserve seeds and candidate architectures, JavaScript Object Notation (JSON) records models and metrics, and NumPy zipped archives (NPZ) store predictions. Every figure is therefore traceable to an array and evaluation protocol.

Installation uses `pip install -e ".[test]"`, while the `[ml]` extra adds the pinned TensorFlow/Keras stack. The `smoke` command verifies the digest and array contract before checking finite values and walls; `qa` applies the complete gate, `figures` regenerates direct-validation plots, and `rom` regenerates the new classical-ROM evidence. Core audits run on a central processing unit (CPU), whereas advanced Fokker–Planck calculations require a graphics processing unit (GPU). The MIT license permits reuse, and a release tag with archival DOI identifies the evaluated software.

2.3. Model choices and validation metrics

The coordinate multilayer perceptron (MLP) maps (Re, x, y) to (u, v, p) with a hyperbolic-tangent activation suited to the smooth cavity family. Complete development fields select among predefined widths and depths, while an output transformation imposes wall velocities and the pressure gauge exactly.

POD–DeepONet combines a fixed spatial basis with a coefficient branch, separating representation error from coefficient-prediction error. Because only Reynolds number varies, a scalar branch suffices and the model remains a restricted parametric operator. Branch width and POD rank are frozen before blind evaluation, while three seeds quantify optimization variability.

Week 4.1 pathway supplies a classical dynamical baseline for the same cavity. Boundary lifting and mean subtraction precede weighted POD. POD–Galerkin projects the full-order evolution onto the retained modes, but its nonlinear term still reconstructs the full state. POD–DEIM constructs a second nonlinear basis and samples convection only at selected interpolation

indices. Rank and DEIM dimension are selected at $Re = 300$ before blind trajectories at $Re = 175, 275$ and 375 are opened. The comparison makes nonlinear cost visible before neural operator learning.

The collision and Fokker–Planck networks solve local maps that are evaluated after deployment in their time-advancing solvers. A stochastic latent input preserves particle variability, while momentum and energy corrections protect invariants. Since the Fokker–Planck MLP is called in every cell, evaluation measures closed-loop stability and end-to-end cost rather than offline accuracy alone; macroscopic fields are therefore examined with higher moments.

2.4. Executable DSMC and neural algorithms

Algorithm 1: validated DSMC cavity. (1) Initialize simulator particles from a Maxwellian distribution and assign a fixed particle weight. (2) Move particles ballistically for one time step and resample molecules striking the four fully diffuse, fully accommodating walls; the upper-wall tangential Maxwellian is shifted by U_{lid} . (3) Sort particles by cell, form the Bird no-time-counter candidate count from the cell population, particle weight, collision majorant, time step and cell volume, and accept each variable-hard-sphere pair with probability $(\sigma g)/(\sigma g)_{max}$. (4) Scatter accepted pairs isotropically in the center-of-mass frame so that pair momentum and kinetic energy are conserved. (5) After the declared transient, pool particle counts and first and second velocity moments over the sampling window before computing density, velocity, temperature and pressure. (6) Repeat with declared seeds and compare wall pressure with literature data before using any particle field as a learning target. The CPU teaching model exposes the same move–wall–collide–sample sequence, whereas the validation run uses the vectorized GPU implementation and records collision diagnostics.

Algorithm 2: bounded neural-surrogate workflow. (1) Verify hashes and numerical acceptance records, then partition complete physical cases. (2) Fit scalars and select architecture, POD rank, loss weights and stopping rules only on development cases. (3) Freeze these choices, retrain on the permitted development set and evaluate predeclared blind cases. (4) Apply exact output transformations where prescribed boundary values are known. (5) Compare every neural result with interpolation or a classical ROM and report fields, local profiles, residuals, constraints and timings rather than training loss alone. The early notebooks instantiate this algorithm with small dense networks and a coordinate MLP; Week 4 uses a three-mode POD trunk and a two-layer 32-neuron coefficient branch; the final research track uses a 16–128–128–128–128–9 Fokker–Planck closure network and judges it

only after closed-loop deployment. This progression teaches the essential logic before exposing research-scale models.

All continuum field errors use the relative discrete L_2 norm

$$\epsilon_2(q) = \left[\frac{\sum_{i=1}^N (q_i^{pred} - q_i^{ref})^2}{\sum_{i=1}^N (q_i^{ref})^2} \right]^{1/2}. \quad (5)$$

Here q is the evaluated scalar field, i identifies one of N locations, and superscripts *pred* and *ref* denote predicted and reference values. Both velocity components enter the norm, whereas pressure fields share one gauge. Pointwise maps reveal where discrepancies occur.

Global norms are paired with local tests because overlap can conceal measurable or corner-localized errors. The continuum pathway combines Ghia centerlines and Botella–Peyret pressure profiles with wall root-mean-square (RMS) error and discrete divergence. DSMC tests add replicate spread, higher moments and maximum pointwise differences.

2.5. Software functionalities

The principal functions are:

- Cavity-field generation includes residual audits, Ghia centerlines and least-squares pressure recovery.
- Case-wise baselines accompany coordinate MLP models; POD–Galerkin and POD–DEIM expose intrusive dynamics and nonlinear cost; POD–DeepONet predicts coefficients for a fixed spatial basis [13, 14, 15].
- Maxwellian sampling and particle-moment recovery introduce sampling analysis before the readable DSMC implementation [16].
- Published continuum and DSMC data are compared through figures whose numerical norms are also stored in machine-readable form.
- Extension interfaces support generalization studies, physics-guided losses, uncertainty analysis and learned closures.
- Deterministic release checks combine dataset hashing with figure generation from saved evidence.

The software deliberately separates compact reference solvers from research-grade evidence. Reference implementations remain small enough to inspect

and modify on a CPU, whereas claims about learned collision or closure models are tied to published a-posteriori studies [17, 8, 18]. This boundary prevents a reduced teaching configuration from being presented as production-converged evidence.

3. Illustrative examples

3.1. *Continuum solver and pressure validation*

Figure 1 assembles the continuum evidence produced by the cavity notebook. At $\text{Re} = 400$, the two nondimensional velocity centerlines are compared with Ghia et al. [4], while the speed field and streamlines provide an independent test of flow topology. Pressure is then evaluated at $\text{Re} = 1000$ on 65^2 and 129^2 grids against Botella and Peyret [5], using the common gauge $p(1/2, 1/2) = 0$. Grid refinement reduces the vertical-centerline relative L_2 error from 18.52% to 5.54% and the horizontal-centerline error from 20.77% to 6.58%. After four grid layers adjacent to the singular moving-lid corners are excluded, the interior pressure-gradient mismatch is 6.62%; the corresponding global value is retained separately so that corner sensitivity remains visible.

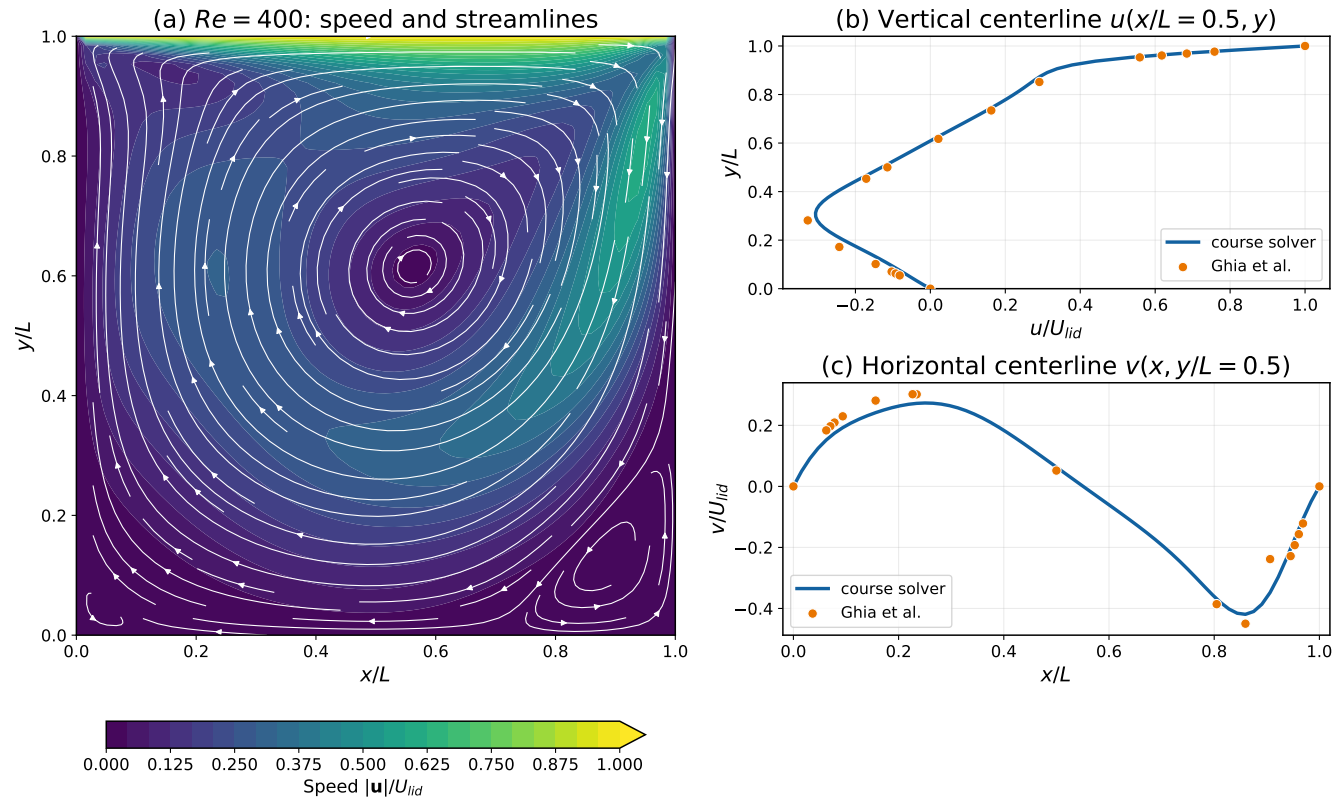


Figure 1: Continuum velocity validation. The $Re = 400$ speed field and streamlines are paired with velocity-centerline comparisons against Ghia et al. [4]. All plotted quantities are nondimensional.

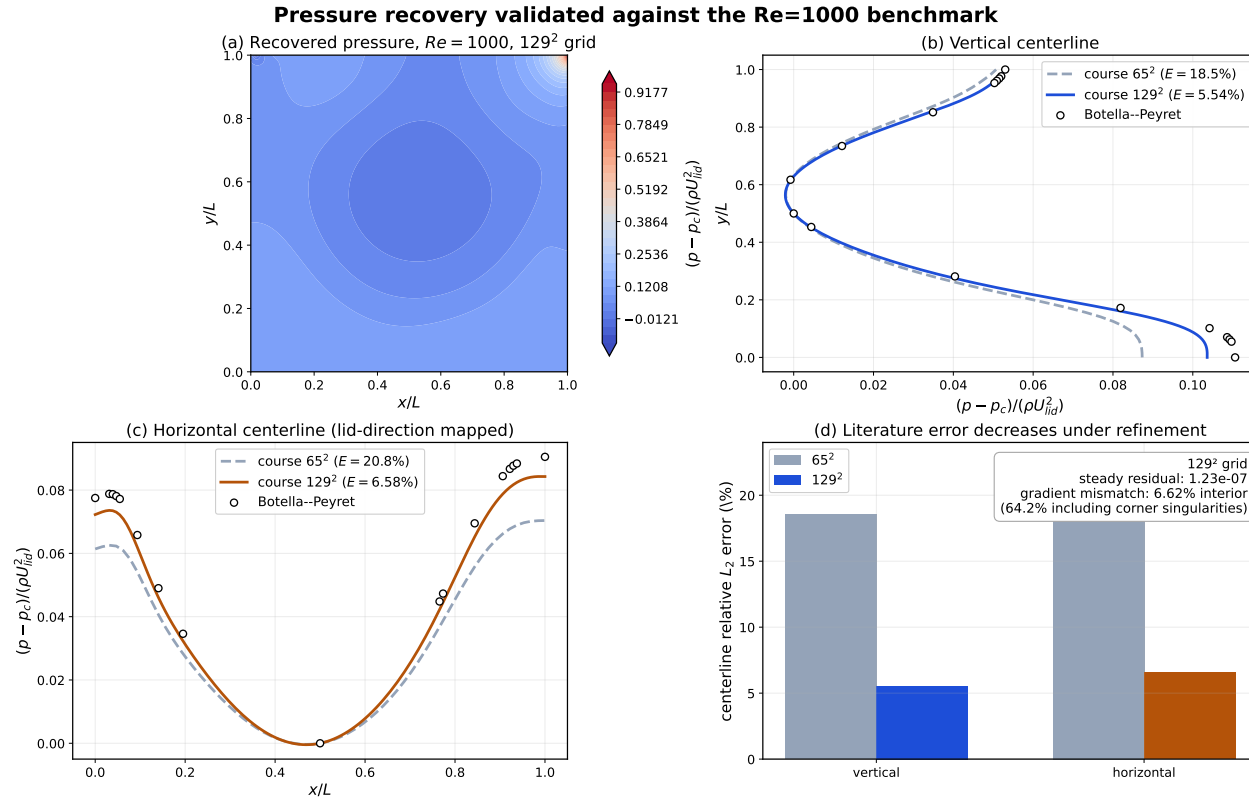


Figure 2: Continuum pressure validation at $Re = 1000$. Botella-Peyret centerlines and grid-refinement errors test pressure separately from the accepted velocity field [5].

3.2. *Coordinate-network validation*

The coordinate MLP maps (Re, x, y) to (u, v, p) after its width, depth and scalers have been selected exclusively from development cases. Its output transformation enforces both wall velocities and the pressure gauge exactly. Figure 3 presents the untouched $\text{Re} = 275$ case, for which the three-seed ensemble attains relative L_2 errors of 1.423% in velocity and 4.264% in pressure. Field-difference maps reveal localized departures that overlapping centerlines can conceal, while interpolation remains the required comparator for deciding whether the neural model adds value.

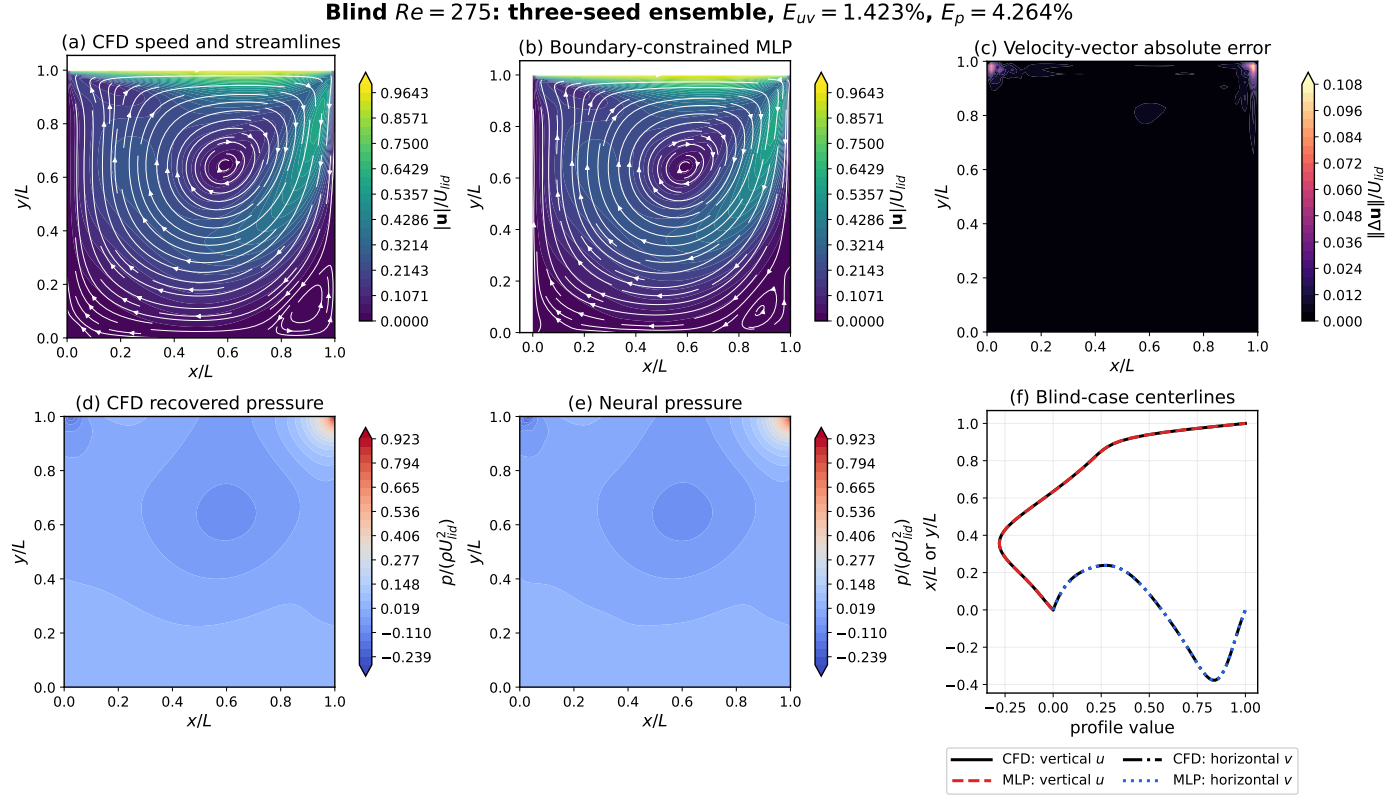


Figure 3: Coordinate-MLP validation on the blind $Re = 275$ cavity. CFD and MLP speed fields share one scale, streamlines test topology, and the vector-error map localizes velocity differences. Recovered and neural pressure also use a common scale, followed by nondimensional centerline comparisons.

3.3. *POD-DeepONet validation and inference advantage*

The POD-DeepONet example is trained on complete Reynolds-number cases rather than randomly partitioned grid points. Development-only selection retains three POD modes and a branch network with two hidden layers of 32 neurons each. Three fixed seeds are then evaluated on untouched fields at $\text{Re} = 175, 275$ and 375 , where the ensemble velocity relative- L_2 errors are 0.453%, 0.0718% and 0.0928%, respectively. An output transformation imposes the exact wall velocity, while the POD basis preserves the discrete divergence structure inherited from the CFD snapshots.

Figure 4 shows that the prediction preserves the centerline fidelity of the reference CFD solver, without claiming to improve the Ghia benchmark itself. Its demonstrated advantage is rapid parameter-to-field evaluation after offline training: a three-seed 65×65 ensemble requires approximately 0.93 ms, whereas the reference CPU solution requires 8.0 s in the measured environment. The resulting inference-to-solver speed ratio is approximately 8.7×10^3 .

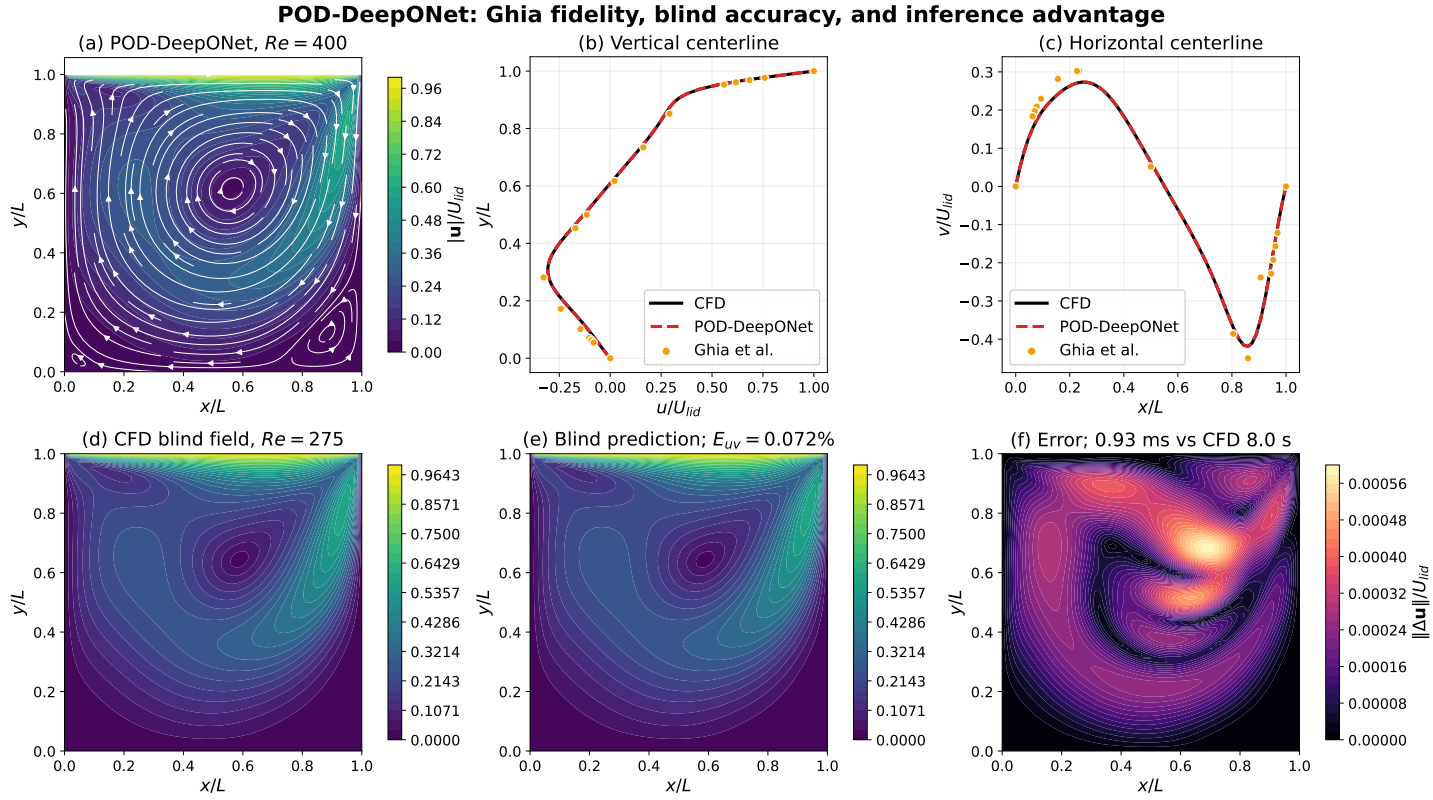


Figure 4: POD-DeepONet validation. Black lines represent CFD, red dashed lines represent POD-DeepONet, and orange markers show Ghia et al. [4]. The lower panels compare the blind $Re = 275$ field with its pointwise velocity-vector error.

3.4. Centered POD–Galerkin and POD–DEIM

Week 4.1 notebook first reproduces the accepted full-order trajectories at $\text{Re} = 100$ and 400 to below 8×10^{-16} relative error, then verifies monotonic grid and time-step refinement. Rank 16 and DEIM dimension 16 are frozen on $\text{Re} = 300$. Across the untouched $\text{Re} = 175, 275$ and 375 trajectories, the maximum velocity error is 0.4941% for POD–Galerkin and 0.6338% for POD–DEIM. Maximum final-vorticity error is 0.5582% , wall error is zero to machine precision, and the maximum divergence norm is 1.43×10^{-16} . POD–Galerkin is not faster than the full-order query ($0.93\times$), whereas POD–DEIM reaches $9.24\times$ online speedup and recovers its offline cost after 7.84 queries. Figure 5 therefore documents both the negative nonlinear-cost result and the hyper-reduced alternative.

FlowMLLab Week 4.1: classical and hyper-reduced ROM validation

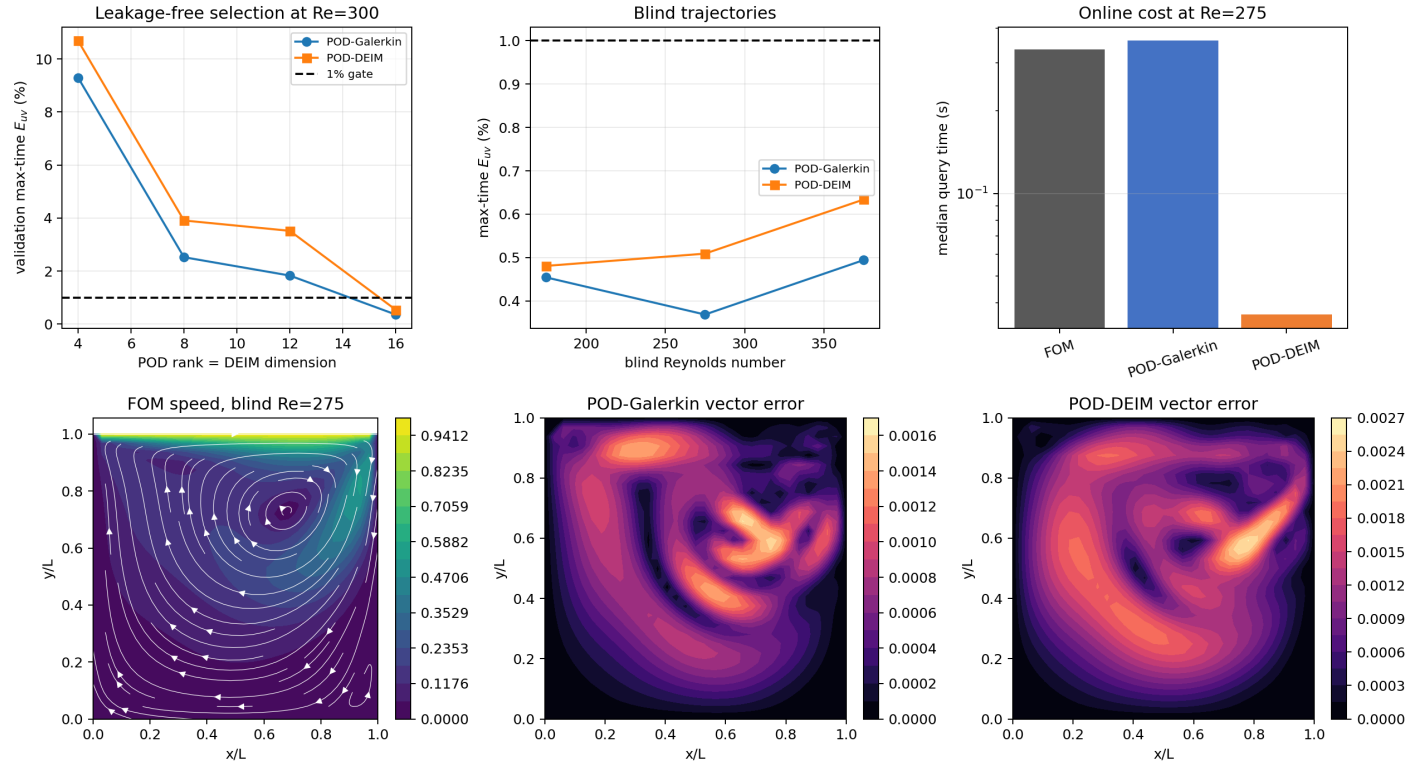


Figure 5: FlowMLLab 1.1.0 classical-ROM evidence for the same cavity. POD-Galerkin and POD-DEIM are compared with the accepted full-order model through selection and blind errors, field structure, nonlinear approximation, timing and break-even cost.

3.5. Direct DSMC validation

The DSMC pathway begins with a literature comparison at $\text{Re} = 1.5$ and $\text{Kn} = \lambda/L = 0.1$, where λ is the molecular mean free path. The Mach number is $\text{Ma} = U_{lid}/a_0 = 0.09$, with a_0 denoting the reference speed of sound. Hard-sphere argon is confined by diffuse, fully accommodating walls at a reference temperature $T_0 = 273 \text{ K}$ and pressure $p_0 = 101135 \text{ Pa}$. Together with the lid speed $U_{lid} = 30.35 \text{ m s}^{-1}$, these values reproduce the published cavity condition.

The solver uses 50 particles per cell and accumulates 6000 field samples before applying a declared five-neighbor pressure filter once. Figure 6 compares nondimensional wall pressure around the closed path A–B–C–D–A with 30 DSMC markers digitized from Mohammadzadeh et al. [6]. The 60^2 mesh gives a relative L_2 difference of 0.772%, while three independent 40^2 samples quantify stochastic spread and the change between the two resolutions provides a grid-sensitivity diagnostic. Literature validation thus precedes every neural DSMC comparison and is performed at the published condition rather than at the wall speed used in the neural-collision study.

HS--NTC solver validation against Mohammadzadeh DSMC wall pressure

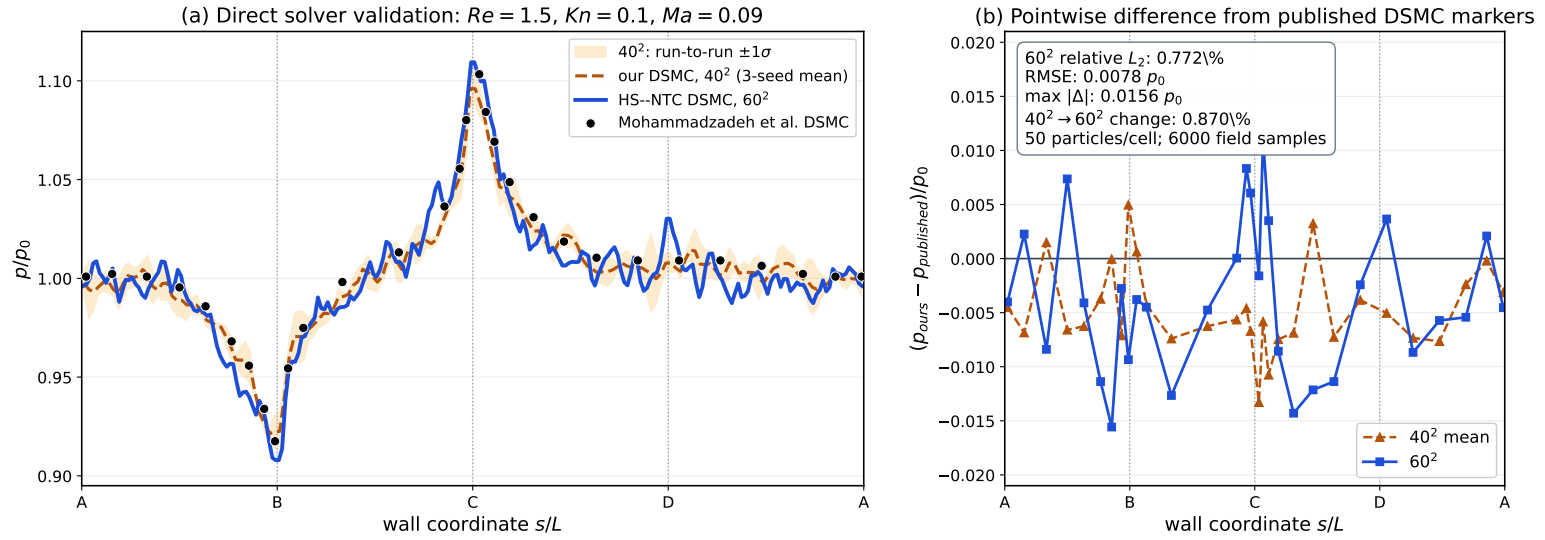


Figure 6: HS-NTC DSMC wall-pressure validation. Blue shows the 60^2 solver result, ochre shows the three-seed 40^2 mean with its spread, and black circles identify the published Mohammadzadeh et al. DSMC values [6].

3.6. Neural-DSMC fields and higher-order moments

The research-transfer example uses a stochastic neural collision operator trained on homogeneous Couette-flow collisions and deployed inside a two-dimensional cavity without geometry-specific retraining [8]. Figure 7 compares the primary fields and centerlines before examining shear stress and heat flux, thereby testing both equilibrium-scale quantities and higher moments. The published relative errors are 0.835% for velocity magnitude and 0.175% for temperature, compared with 1.56% for shear stress and 1.48% for heat flux. These higher-order comparisons are essential because agreement in velocity and temperature alone cannot establish kinetic fidelity.

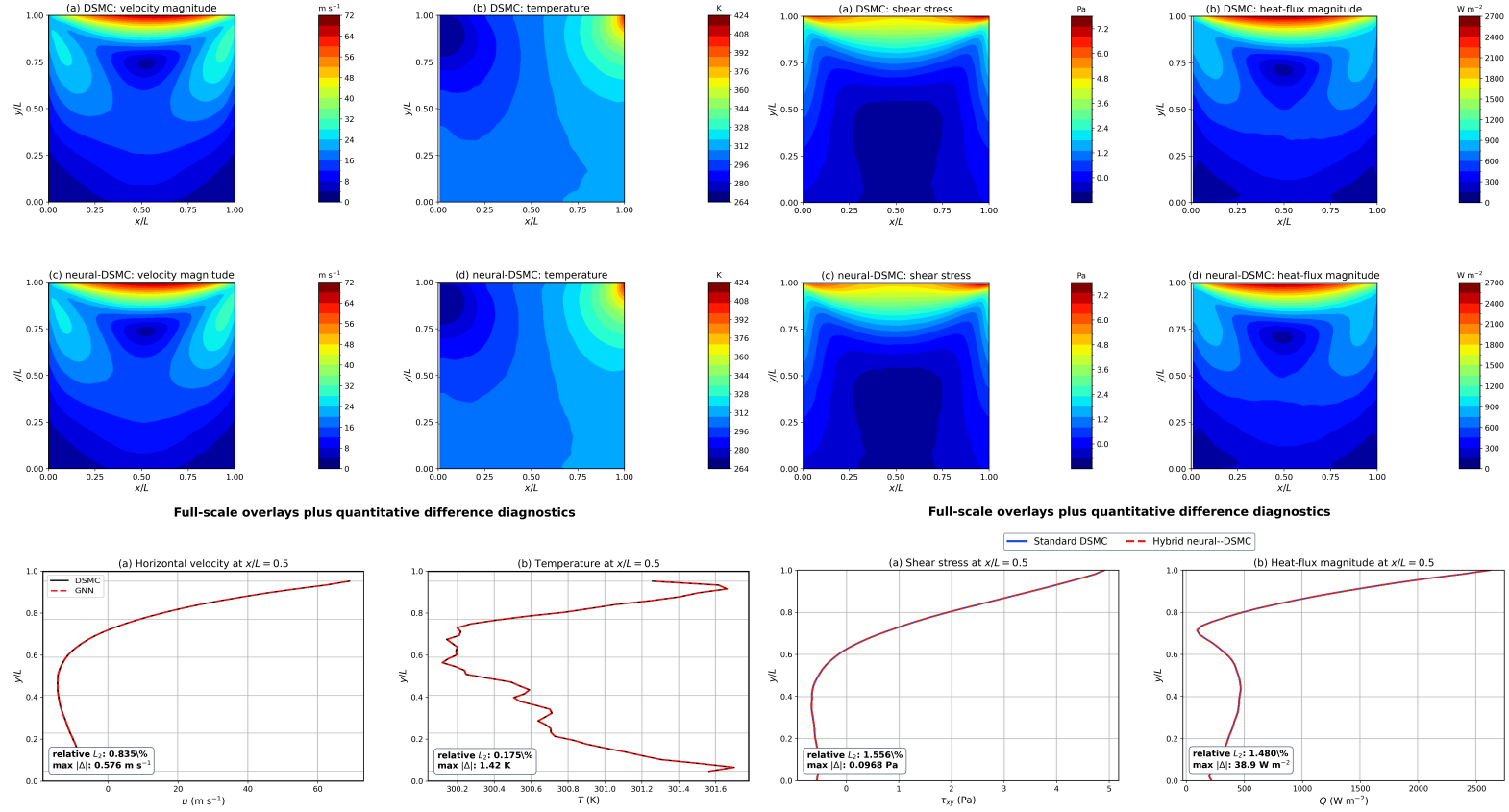


Figure 7: Neural-DSMC validation against standard DSMC [8]. The left column compares velocity magnitude and temperature, while the right column examines shear stress and heat-flux magnitude. Field maps occupy the upper row and the corresponding centerline diagnostics occupy the lower row. In the profile panels, the solid DSMC curves are compared with dashed neural-DSMC curves; the “GNN” label in the primary-profile source figure refers to the same graph-neural collision operator. Quantitative boxes report relative L_2 and maximum pointwise differences.

3.7. Closed-loop Fokker–Planck validation

The advanced track maps 16 local low-order features to nine closure coefficients and evaluates the learned closure inside the particle solver rather than against an offline coefficient table alone. Figures 8 and 9 compare exact cubic Fokker–Planck fields with machine-learning-assisted Fokker–Planck (ML–FP) fields and their centerlines. Weighting the training loss by the heat-flux magnitude q yields $2.017\times$ online acceleration with a maximum macroscopic relative- L_2 error of 4.6651×10^{-2} . The high-heat-flux coefficient statistic improves by 2.92442% relative to uniform-loss training, following the published use of neural surrogates for accelerating cubic Fokker–Planck closures [18].

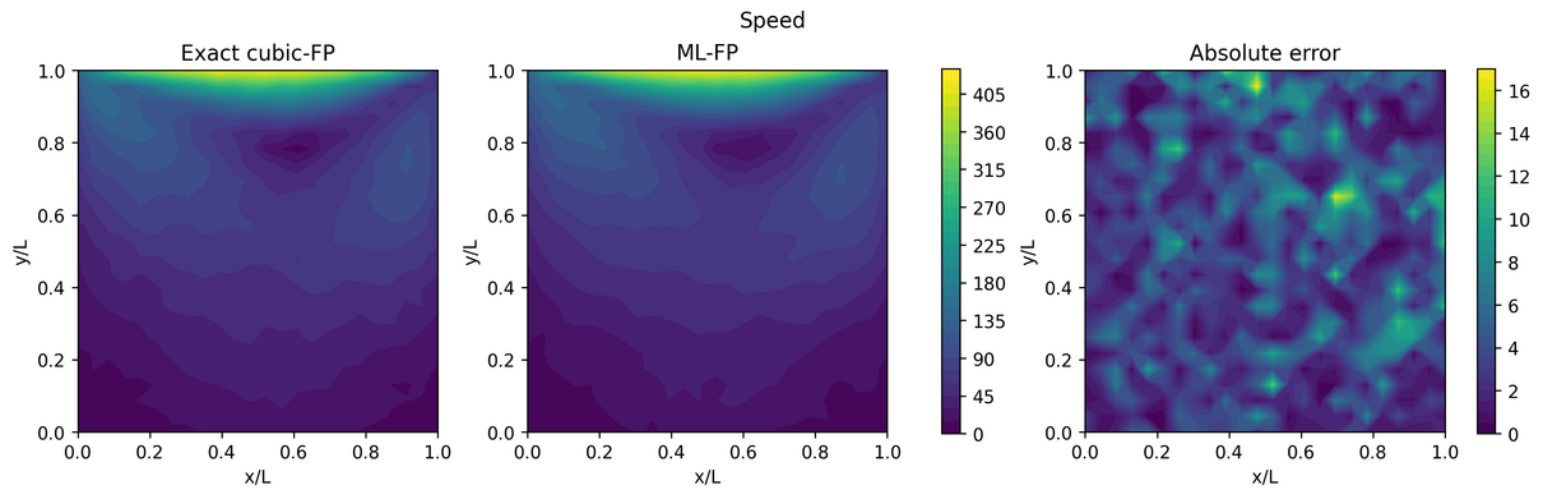


Figure 8: Closed-loop Fokker-Planck field validation. Exact cubic-closure and ML-FP speed fields are shown with their absolute error.

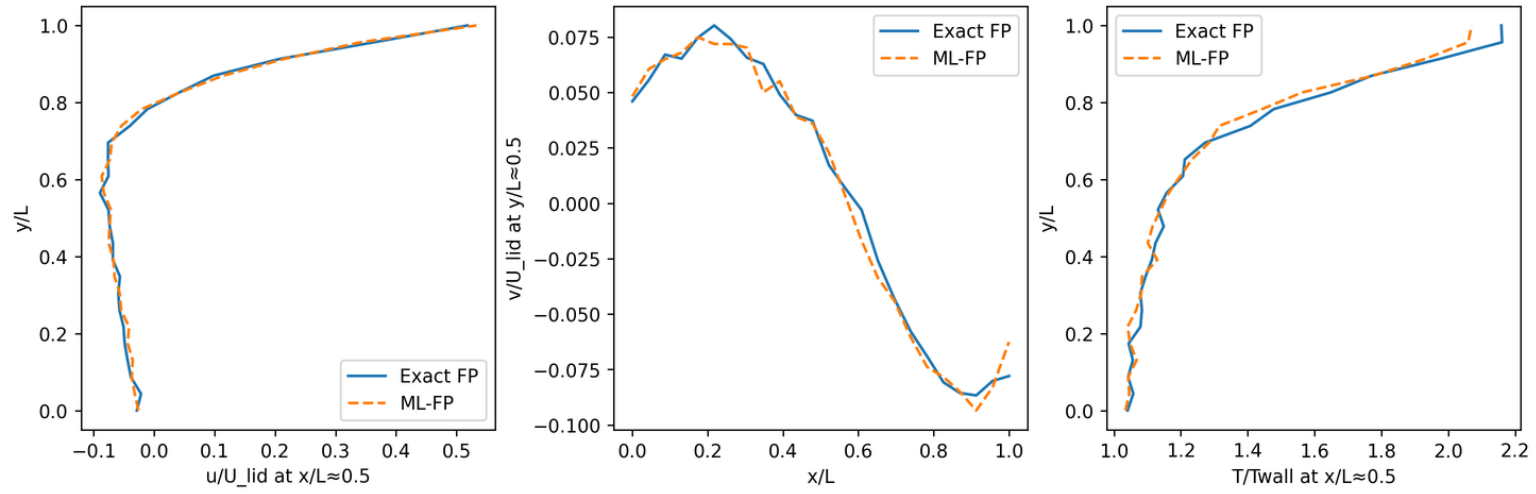


Figure 9: Closed-loop Fokker-Planck profile validation. Nondimensional centerline velocities and T/T_{wall} are compared, where T is local temperature and T_{wall} is wall temperature. Blue represents the exact closure and orange dashed lines represent ML-FP.

4. Impact

FlowMLLab contributes an experimental harness rather than another neural architecture. A versioned code base connects solvers, benchmark data, case-wise partitions, non-neural baselines, predictions and timings. Evaluation never rests solely on training loss: continuum cases require near-wall velocity, pressure, walls and divergence; particle cases require replicate variation and higher moments; closure models require stable closed-loop deployment. Negative results remain visible, as illustrated by POD–Galerkin failing to accelerate the nonlinear query. Online timing is reported separately from offline data and training cost.

4.1. Scientific applications and extensibility

Continuum and kinetic components share the same protocol while exposing different numerical risks. New surrogates consume declared case lists without modifying dataset verification, baselines or metrics. A new physical case must define nondimensional parameters, its array schema, a quality record and an external-validation target; a kinetic closure must additionally expose conserved quantities and its correction step. Hashes detect deterministic data substitution, whereas seed-specific records and ensemble summaries distinguish normal DSMC variation from a software defect. Related research applications include rarefied-flow neural models, collision operators, shock-aligned representations and closure acceleration [17, 8, 19, 18].

4.2. Tutorials and educational reuse

The 17 notebooks call the same public modules as the CLI instead of maintaining parallel implementations. The five linked lectures progress from numerical foundations (Week 1), supervised learning and rarefaction (Week 2), and DSMC (Week 3) to cavity surrogates (Week 4) and bounded research projects (Weeks 5–6). Optional Week 4.1 places classical POD dynamics and hyper-reduction immediately before POD–DeepONet. Thus a graduate learner or industry engineer encounters essential concepts first, while advanced operators and closures remain later extensions.

4.3. Limitations and future development

The continuum archive is a two-dimensional square cavity in which only Reynolds number varies; it does not establish turbulent or complex-geometry capability. Pressure remains resolution sensitive near the lid corners, and the scalar Reynolds-number branch is a restricted parametric operator. The

readable VHS–NTC solver lacks production domain decomposition and broad collision-model support, while its uncertainty depends on particle count and averaging. Core audits and ROMs run on a CPU, but optional TensorFlow and Fokker–Planck workflows remain hardware dependent. Future work should add geometry and boundary parameters, uncertainty calibration, out-of-distribution detection, and kinetic benchmarks spanning several Knudsen and Mach numbers.

5. Conclusions

FlowMLLab provides a reproducible path from numerical fluid mechanics to validated scientific machine learning under a contract governing solver output, data qualification, case-wise partitions and baselines. Physical diagnostics and machine-readable evidence make surrogate checks available to users and continuous integration. The continuum examples validate velocity and pressure before blind-field assessment, while POD–DeepONet demonstrates an inference advantage without overstating benchmark accuracy. DSMC receives direct literature validation before neural collision transfer, and Fokker–Planck is tested in closed-loop operation. Tutorials make these capabilities accessible, but each claim remains tied to reusable code, an immutable record, a matched baseline and a physical validation target.

Software, tutorials, and documentation links

The links below provide direct access to the framework, tutorial notebooks and supporting documentation on the `main` branch, while Table 1 identifies the immutable software version evaluated in this article.

- [Complete FlowMLLab software tree](#)
- **Week 1 notebooks:** [Python for fluid simulation and scientific machine learning](#); [TensorFlow foundations](#); [cavity velocity and pressure validation](#); [Week 1 lecture PDF](#).
- **Week 2 notebook:** [supervised learning](#); [baselines](#); [rarefaction](#); [Week 2 lecture PDF](#).
- **Week 3 notebooks:** [Maxwellian moments and sampling noise](#) and [DSMC cavity and Mohammadzadeh validation](#); [Week 3 lecture PDF](#).
- **Week 4 notebooks:** [CFD data production and audit](#); [scalar and field surrogates](#); [POD–DeepONet cavity](#); [Week 4 lecture PDF](#).

- **Week 4.1 addition:** [centered POD–Galerkin, nonlinear-cost diagnosis and POD–DEIM hyper-reduction for the same cavity](#); all earlier lectures and notebooks remain unchanged.
- **Weeks 5–6 setup and tracks:** [project setup](#); [Reynolds-number generalization](#); [physics-guided deep neural network](#); [proper-orthogonal-decomposition study](#); [uncertainty study](#); [rarefied cavity](#); [Fokker–Planck closure](#); [Weeks 5–6 project guide PDF](#).
- [Shared CFD utilities](#); [pressure recovery](#); [POD–DeepONet](#); [DSMC](#); [QA](#)
- [Fixed cavity dataset and case-quality table](#)
- [Validation data](#); [predictions](#); [metrics](#); [timings](#); [article figures](#)
- [Advanced Fokker–Planck closure software](#)
- [Notebook builders](#); [synchronization utilities](#); [release validator](#)
- [Annotated reading guide and BibTeX references](#)

Acknowledgements

No external funding was received for development of the software described in this article.

Contributor Roles Taxonomy (CRediT) authorship contribution statement

Ehsan Roohi: Conceptualization; Methodology; Software; Validation; Investigation; Resources; Data curation; Writing–original draft; Writing–review and editing; Visualization; Supervision; Project administration.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the writing process

During preparation of this work, the author used a large language model to improve language, organization and LaTeX presentation. The author reviewed and edited the output and takes full responsibility for the content of

the publication. No AI system generated or replaced the reported numerical evidence.

Data and code availability

The permanent GitHub tag in Table 1 contains the installable package, 17 tutorial notebooks, fixed datasets, validation records and continuous-integration workflow. Figure and ROM evidence are preserved in the version-specific Zenodo record [10.5281/zenodo.22126785](https://doi.org/10.5281/zenodo.22126785); direct links to every tutorial and lecture are provided above.

References

- [1] Steven L. Brunton, Bernd R. Noack, and Petros Koumoutsakos. Machine learning for fluid mechanics. *Annual Review of Fluid Mechanics*, 52:477–508, 2020. doi: [10.1146/annurev-fluid-010719-060214](https://doi.org/10.1146/annurev-fluid-010719-060214).
- [2] Karthik Duraisamy, Gianluca Iaccarino, and Heng Xiao. Turbulence modeling in the age of data. *Annual Review of Fluid Mechanics*, 51: 357–377, 2019. doi: [10.1146/annurev-fluid-010518-040547](https://doi.org/10.1146/annurev-fluid-010518-040547).
- [3] Steven L. Brunton. Applying machine learning to study fluid mechanics. *Acta Mechanica Sinica*, 37:1718–1726, 2021. doi: [10.1007/s10409-021-01143-6](https://doi.org/10.1007/s10409-021-01143-6).
- [4] U. Ghia, K. N. Ghia, and C. T. Shin. High-re solutions for incompressible flow using the navier–stokes equations and a multigrid method. *Journal of Computational Physics*, 48(3):387–411, 1982. doi: [10.1016/0021-9991\(82\)90058-4](https://doi.org/10.1016/0021-9991(82)90058-4).
- [5] O. Botella and R. Peyret. Benchmark spectral results on the lid-driven cavity flow. *Computers & Fluids*, 27(4):421–433, 1998. doi: [10.1016/S0045-7930\(98\)00002-4](https://doi.org/10.1016/S0045-7930(98)00002-4).
- [6] Alireza Mohammadzadeh, Ehsan Roohi, Hamid Niazmand, Stefan Stefanov, and Rho Shin Myong. Thermal and second-law analysis of a micro- or nanocavity using direct-simulation monte carlo. *Physical Review E*, 85:056310, 2012. doi: [10.1103/PhysRevE.85.056310](https://doi.org/10.1103/PhysRevE.85.056310).
- [7] Simon Mizzi, David R. Emerson, Stefan K. Stefanov, Robert W. Barber, and Jason M. Reese. Effects of rarefaction on cavity flow in the slip regime. *Journal of Computational and Theoretical Nanoscience*, 4(4): 817–822, 2007. doi: [10.1166/jctn.2007.2374](https://doi.org/10.1166/jctn.2007.2374).

- [8] Ehsan Roohi, Ahmad Shoja-Sani, and Stefan Stefanov. Physics constrained neural collision operators for hard sphere surrogates and ab initio angle prediction in direct simulation monte carlo. *Physics of Fluids*, 38:057123, 2026. doi: [10.1063/5.0328463](https://doi.org/10.1063/5.0328463).
- [9] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. Jupyter notebooks – a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90. IOS Press, 2016. doi: [10.3233/978-1-61499-649-1-87](https://doi.org/10.3233/978-1-61499-649-1-87).
- [10] Lorena A. Barba and Gilbert F. Forsyth. CFD Python: the 12 steps to Navier–Stokes equations. *Journal of Open Source Education*, 1(9):21, 2018. doi: [10.21105/jose.00021](https://doi.org/10.21105/jose.00021).
- [11] Greg Wilson, D. A. Aruliah, C. Titus Brown, Neil P. Chue Hong, Matt Davis, Richard T. Guy, Steven H. D. Haddock, Kathryn D. Huff, Ian M. Mitchell, Mark D. Plumbley, Ben Waugh, Ethan P. White, and Paul Wilson. Best practices for scientific computing. *PLoS Biology*, 12(1): e1001745, 2014. doi: [10.1371/journal.pbio.1001745](https://doi.org/10.1371/journal.pbio.1001745).
- [12] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013. doi: [10.1371/journal.pcbi.1003285](https://doi.org/10.1371/journal.pcbi.1003285).
- [13] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021. doi: [10.1038/s42256-021-00302-5](https://doi.org/10.1038/s42256-021-00302-5).
- [14] Gal Berkooz, Philip Holmes, and John L. Lumley. The proper orthogonal decomposition in the analysis of turbulent flows. *Annual Review of Fluid Mechanics*, 25:539–575, 1993. doi: [10.1146/annurev.fl.25.010193.002543](https://doi.org/10.1146/annurev.fl.25.010193.002543).
- [15] Jan S. Hesthaven and Stefano Ubbiali. Non-intrusive reduced order modeling of nonlinear problems using neural networks. *Journal of Computational Physics*, 363:55–78, 2018. doi: [10.1016/j.jcp.2018.02.037](https://doi.org/10.1016/j.jcp.2018.02.037).
- [16] G. A. Bird. *Molecular Gas Dynamics and the Direct Simulation of Gas Flows*. Oxford University Press, 1994. doi: [10.1093/oso/9780198561958.001.0001](https://doi.org/10.1093/oso/9780198561958.001.0001).

- [17] Ehsan Roohi, Ahmad Shoja-Sani, and Fahimeh Ebrahimzadeh Azghadi. Neural networks for rarefied gas dynamics: Relaxation problem, polyatomic shock waves, and hypersonic cylinder flow. *Physics of Fluids*, 38: 057108, 2026. doi: [10.1063/5.0334590](https://doi.org/10.1063/5.0334590).
- [18] Ehsan Roohi. Accelerating kinetic fokker–planck simulations via a gpu-native deep neural network surrogate: Application to rarefied internal and hypersonic external flows. *Journal of Computational Physics*, 566: 115261, 2026. doi: [10.1016/j.jcp.2026.115261](https://doi.org/10.1016/j.jcp.2026.115261).
- [19] Ehsan Roohi and Amirmehran Mahdavi. Shock-centered low-rank structure and shock-aligned surrogate modeling of rarefied micro-nozzle flows. *Physics of Fluids*, 38(8):082008, 2026. doi: [10.1063/5.0343101](https://doi.org/10.1063/5.0343101).

Current executable software version

FlowMLLab 1.1.0 is the tagged commit identified in Table 1; its declared dependencies and command-line interface are tested on Python 3.10–3.12.